# Multi-Bottleneck Fairness in RDMA Fabrics: Measurement and an Explicit-Rate Service Mode

How deployable RDMA congestion control behaves under multi-bottleneck contention, why sender-side correction stalls, and RDMA-RCP over in-network telemetry

Haoyu Wang
University of Massachusetts Boston
Boston, MA, USA
haoyu.wang001@umb.edu

Bo Sheng
University of Massachusetts Boston
Boston, MA, USA
bo.sheng@umb.edu

Zerin Shaima Meem
University of Nebraska Omaha
Omaha, NE, USA
zmeem@unomaha.edu

Xiaoqian Zhang
University of Nebraska Omaha
Omaha, NE, USA
xiaoqianzhang@unomaha.edu

## Abstract

Modern datacenter traffic increasingly crosses several contended links in series, yet deployable RDMA congestion control is evaluated almost exclusively on single-bottleneck benchmarks. We present an empirical study of multi-bottleneck fairness on lossless RDMA fabrics. On a parameterized parking-lot benchmark with a fluid max-min oracle, *every* deployable scheme we test is unfair to the multi-bottleneck flow — DCQCN 1.7–2.3×, TIMELY 1.8–2.0×, DCTCP 1.6–1.8×, HPCC 1.9–2.0× — growing with bottleneck count and robust to start-time jitter. For HPCC we localize the mechanism: near-winner-take-all collapse to ∼0.4 Gbps. Six sender-only corrections fail — including a sender-side mirror of RCP driven by the same INT signals: independently maintained copies of an explicit-rate recursion preserve arrival-history asymmetry indefinitely, while the same recursion on one shared per-link register erases it. Per-flow fair queueing equalizes only what the endpoint offers: 1.97× windowed, equal-but-idle (∼13× oracle FCT) windowless. Placing that register at the switch — RDMA-RCP, an RCP service mode carried in one 64-bit INT field with three words of mutable state per port — restores the max-min allocation (penalty ≈1.005×; flows within 5–13% of oracle FCT across the heterogeneous matrix, a uniform 21–39% under three-factor compound stress) with zero PFC in every fairness experiment, rate-only or with a canonical $cwnd = R \cdot RTT$ window, and cuts a ring-collective coflow's completion time by 18–23% at every seed tested (18% with the INT header padded to stock size). We equally measure the deployment boundary: the modes do not share an unsliced link fairly — even under weighted-DRR reservation, both control signals remain port-global — so RDMA-RCP is a fabric-wide or isolated-slice mode. Stock HPCC's code remains byte-for-byte unchanged.

## 1 Introduction

HPCC [15] and its derivatives represent the state of the art in datacenter RDMA congestion control: the switch stamps raw per-hop state into INT headers, and the sender computes a precise utilization estimate toward a target $\eta$, matching or beating DCQCN, TIMELY, and DCTCP on the canonical single-bottleneck benchmarks — which is where deployable RDMA congestion control is, almost exclusively, evaluated.

Modern datacenter traffic, however, increasingly crosses *several contended links in series*: cross-pod ML collectives, disaggregated memory and storage, multi-tenant aggregation and core layers. The practical stake is concrete: a collective completes when its *slowest* flow completes, and the slowest flows are precisely the cross-fabric, multi-bottleneck ones. On a 16-flow ring-collective coflow over a $k$=4 fat-tree, we measure stock HPCC inflating job completion time by ∼27% over what max-min allocation delivers — at every ECMP hash seed we test (§5.4). Whether deployable RDMA congestion control is fair to such traffic has not been carefully measured. This paper measures it.

Max-min fairness is a policy choice, not a universal congestion-control objective; our claims target traffic classes whose completion is gated by multi-bottleneck flows, and §5.14 delimits where it is the wrong target. We proceed with a deterministic, oracle-grounded methodology. Our simulator *is* the original HPCC authors' ns-3 code base, upgraded to a tagged modern release (ns-3.45): the changes are interface-level — build-system, header, and deprecated-API adaptations — while the congestion-control logic, switch datapath, and wire formats are the original code (one latent field-initialization bug, exposed only by the 376-node topology, is fixed and documented). We validated the upgrade by reproducing the original paper's baseline comparisons across five schemes and two production workloads (§5); the reproduction record and raw results are public. The measurements here are therefore produced by the original HPCC implementation's logic, not a re-implementation. Our findings:

(1) **The failure spans every scheme we evaluate, and we characterize HPCC's instance** (§2, §5.3). Every deployable scheme we test is unfair to the multi-bottleneck flow — DCQCN 1.70–2.32×, TIMELY 1.83–2.01×, DCTCP 1.61–1.78×, HPCC 1.90–1.96× at $N = 2$–4 — all growing with bottleneck count, and robust to start-time jitter. For HPCC we isolate the mechanism: the failure is *near-winner-take-all*

(the long flow collapses to ∼0.4 Gbps against ∼23 Gbps competitors; 3.5× for small flows), RTT-independent, immune to HPCC's own per-hop multi-rate mode and to INT compression (PINT), and compounded past four bottlenecks by a wire-format cap (`maxHop` = 5) that corrupts the control input. A single-bottleneck control ($N$=1: 1.10–1.39× pairwise spread, direction set by arrival order) separates HPCC's baseline rate lottery from the systematic multi-bottleneck penalty on top.

(2) **Private replication of the fair-rate computation cannot close the gap — shared placement can** (§3). We evaluate six sender-only designs: five heuristics, plus the most direct variant — a full sender-side mirror of the RCP recursion driven by the same INT-visible inputs the switch would use. The mirror still fails (1.56× even with synchronized starts; 1.65×/0.81× under staggered arrival, the *direction* set by arrival order), while the identical law at the switch holds ≈1.0×; per-flow fair queueing equalizes only what senders offer — windowed HPCC leaves 1.97×, windowless HPCC equalizes at ∼13× the oracle FCT. The mechanism is history: independently maintained rate state preserves arrival-order asymmetry indefinitely, while a shared per-link register eliminates that history dependence — evidence about explicit-rate state placement, not a formal impossibility for every endpoint law (§3).

(3) **An RCP service mode over INT restores max-min, verified end to end** (§4, §5). RDMA-RCP carries a switch-computed RCP fair rate in one 64-bit INT field with three words of mutable state per egress port, alongside byte-for-byte unchanged stock HPCC. It reaches penalty ≈1.005× across bottleneck counts, asymmetric sizes, staggered and jittered starts, and ECMP hash seeds, with zero PFC in every fairness experiment — operating rate-only or with a canonical RCP window ($cwnd = R \cdot \text{baseRTT}$), and cutting ring-collective coflow JCT by 18–23% at every seed tested (18% under a byte-identical INT header). We equally report its deployment boundary: sharing a link with stock HPCC serves neither class, even under a weighted-DRR reservation, because both controllers read port-global telemetry (§4.3) — a fabric-wide or isolated-service mode. The mechanism is RCP; we claim the measurement study and the placement result, not a new control law.

*Artifacts.* The implementation, the topology/workload generators, the max-min oracle, and deterministic scripts that regenerate the figures (`make_figures.py`) and the ablation/sensitivity/baseline tables (`run_sensitivity.py`) from source are available in our artifact-review repository.[1]

# 2 Background and Motivation

## 2.1 HPCC, briefly

HPCC's per-hop utilization estimate for hop $i$ is

$$u_i = \frac{txRate_i}{C_i} + \frac{\min(qlen_i, qlen_i^{\text{prev}}) \cdot maxRate}{C_i \cdot W} \tag{1}$$

where the first term reflects bandwidth utilization and the second is a queue-drain term that keeps standing queues bounded. In single-rate mode the sender takes $U = \max_i u_i$, smooths it over the base RTT, and applies

$$new\_rate = \begin{cases} curRate/\max c + R_{AI} & \max c \geq 1 \\ curRate + R_{AI} & \text{otherwise} \end{cases}$$

with $\max c = U/\eta$. The window $W$ is sized to $m_{win} \cdot rate/NIC\_line$ under `var_win`.

## 2.2 The multi-bottleneck parking lot

The *parking lot* is a classic congestion-control fairness benchmark [6, 12]. Because every link is equally contended, the *max-min fair* allocation [6] gives every flow an equal share, so the long flow should run exactly as fast as each cross flow despite traversing many contention points rather than one. The parking lot therefore isolates the multi-bottleneck fairness question in its simplest form, with no confounding asymmetry in link capacity or hop count of the competitors. One control is essential before reading any deviation as a multi-bottleneck effect: on a *single* shared bottleneck ($N$=1) with two symmetric 50 MB flows, stock HPCC already allocates unequally (37.20 vs 26.72 ms, a 1.39× spread), while RDMA-RCP equalizes them exactly (33.89 vs 33.89 ms). Stock HPCC therefore carries a baseline rate unfairness between symmetric flows; the multi-bottleneck penalty we characterize is the *additional* deviation on top of it, growing to 1.90–1.96× at $N$ = 2–4.

We construct a parameterized parking lot with $N$ inter-switch bottlenecks (25 Gbps) and 100 Gbps host links (Figure 1): one "long" flow crosses all $N$, one "cross" flow crosses each; 50 MB each, simultaneous start, so the max-min rate is 12.5 Gbps for every flow. All experiments are deterministic (byte-identical artifacts across repeated runs, verified).

*Metric.* We compare each flow's actual flow completion time (FCT) against an idealized **fluid max-min oracle** — a continuous bandwidth-sharing model with no protocol dynamics (no slow start, no queueing, no convergence delay) in which the bandwidth is allocated max-min fairly at every instant. We compute the allocation by *progressive filling* [6]: all active flows raise their rate in lockstep until some link saturates; the flows crossing that link are frozen at their current rate; and the process repeats on the remaining flows until every flow is frozen. Because flows complete and depart over time, we run it *event-driven* — recomputing the allocation at each flow completion — so the oracle FCT accounts for bandwidth freed as flows finish; it is exact for our fluid topologies. The headline metric is the **unfairness ratio** $\max_{\text{path}} FCT/\text{mean}(\text{short-path FCT})$; the oracle's value is 1.0 for equal-size, simultaneous flows.

## 2.3 Findings F1–F4: characterizing the gap

*Magnitude.* Stock HPCC's unfairness ratio at $N \in \{2, 3, 4\}$ is 1.90, 1.92, 1.96: the multi-bottleneck flow's FCT is roughly twice that of single-bottleneck competitors. **There is a non-monotonic regime change at** $N \geq 5$, where the long flow becomes favored (0.51×); this turns out to be an INT-overflow artifact at `maxHop` = 5 and

[1] https://github.com/UNOCourseDemo/hpcc-fs

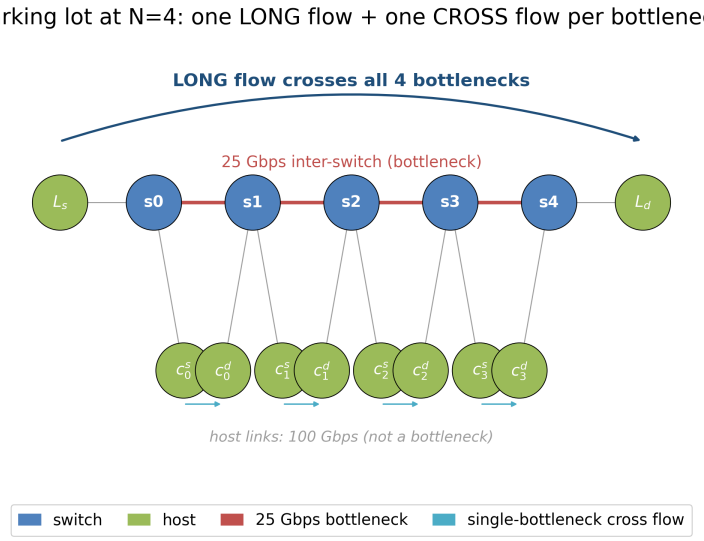



**Figure 1: Parking-lot benchmark at $N = 4$. The long flow $L_s \to L_d$ crosses every bottleneck; each cross flow $c_i^s \to c_i^d$ traverses exactly one. Host links are not bottlenecks.**

we exclude $N \geq 5$ from the rest of this paper's stock comparisons (Figure 2).[2]

*Robustness.* Size and start-time perturbations give 1.81–3.53× (Figure 4), worst (3.5×) when the multi-bottleneck flow is small — the latency-sensitive RPC case. *Multi-rate HPCC* (per-hop EWMA, $\min_i Rc_i$) helps but does not fix it (1.44–1.75×, growing with $N$). *RTT-equalization rules out RTT bias:* the penalty stays ~1.9× across a 4.5× sweep of cross-flow RTTs (cross-flow RTT up to 1.8× the long flow's).

*Trace diagnosis.* At $N = 4$ (Figure 5 left) the long flow collapses to ~0.38 Gbps while the four cross flows hold ~23.2 Gbps each for the entire 19 ms contention window, with zero PFC and zero loss — *near-winner-take-all*: at every shared link the local single-bottleneck flow wins and the long flow is squeezed. This is an equilibrium, not a transient: with 500 MB flows (10× the contention, ~7,000 RTTs) the ratio is 1.975×, the long flow flat at 0.39 Gbps for 150 ms.

## 3 The Limits of Sender-Side Control

A natural first instinct — one we explored exhaustively — is to keep all state at the sender, preserving HPCC's stateless switch. Six sender-only variants sit behind a new `mb_mode` knob (`mb_mode = 0` byte-identical to stock, verified): five heuristic changes to `UpdateRateHp`, and the most direct — a full sender-side mirror of the RCP recursion.

[2]The 5-hop cap is HPCC's original wire format, not an artifact of our ns-3 upgrade: the reference simulator hard-codes `maxHop = 5` in its INT header [2], consistent with the paper's 42-byte worst-case INT overhead (five 8-byte hop records). Conversely, a parking-lot path at $N \leq 4$ traverses at most five switches — *within* the INT budget — so the retained $N = 2$–4 measurements are structurally free of the overflow.

### 3.1 Five heuristic variants

The five heuristics attack different levers of HPCC's update law: a $k$-aware additive boost scaled by the congested-hop count (1.63×, the best case, at $\gamma$=50); a debiased max/mean blend of per-hop $u_i$ (1.95× — the $u_i$ are nearly equal here); a responsibility-weighted MD (1.89×) and a $k$-th-root MD (1.92×) — the MD branch barely fires at $\eta$-hold; and a share-aware additive increase on every flow (1.71×). Stock is 1.96×; the target is 1.0.

### 3.2 Isolating placement: identical observations, private vs. shared state

We isolate placement in a deterministic fluid model of one link: every controller receives identical $(y, q)$ at identical instants — same rule, gains, initialization; only the *location of the recursion state* differs (`run_consistency.py`; flow 2 arrives at $t$=5 ms). Shared register: the late arrival inherits the evolved state, 12.5/12.5 Gbps. Private, synchronized: equally fair — information suffices. Private, staggered: the newcomer initializes fresh, both multiply identically thereafter, and the ratio freezes at exactly 2:1. A sender cannot replicate the register's *history*.

### 3.3 The full stack: a virtual-RCP mirror

The fluid model isolates the mechanism; `mb_mode 6` confirms it in the full stack: the sender runs the switch's recursion *privately* per hop from its own INT samples of $C$, $y$, $q$ (identical rule and gains, $R_0 = C/2$, its base RTT as interval), sending at $\min_i \hat{R}_i$. The real stack adds per-flow sampling and feedback-delay asymmetry beyond the fluid model's arrival asymmetry — why the mirror falls short even synchronized.

The mirror reaches only 1.560× synchronized and 1.654×/0.813× with the long/cross side arriving 5 ms late (direction set by arrival

order), while the identical law at the switch holds 1.003–1.008× throughout.

## 3.4 The structural reason: consistency, not information

Two observations explain the pattern. *First, the $\eta$-hold equilibrium:* once the contended links settle at target utilization $\eta$, HPCC's multiplicative term $1/\max c \approx 1$ and every flow holds its current rate — whoever grabbed more at startup keeps it. Additive nudges (modes 1, 5) are orders of magnitude too slow relative to flow lifetimes (200 ms $\approx 10^4$ RTTs, no convergence); MD-side modifications (modes 3, 4) rarely fire at the hold point.

*Second — what mode 6 isolates — private explicit-rate state preserves history.* The recursion's raw inputs are sender-visible (exactly in the fluid model; sampled in the stack); what a private copy cannot reproduce is the register's evolved *history*. A multiplicative update on independently initialized state preserves the ratio set at arrival forever (§3.2); the same update on one shared per-link register makes every flow — incumbent or newcomer — adopt the same $R$ within one RTT, eliminating the history dependence. This is why RCP [9, 10] and XCP [12] place the computation in the network. The scheduling side sharpens the picture (`run_round4.py`): per-flow equal-weight DRR at every port under unchanged (windowed) HPCC leaves 1.97× — the NIC-scaled window underfeeds the long flow, and a work-conserving scheduler cannot allocate service never offered. Removing the window lets DRR equalize (0.997×) — but at ∼0.9 Gbps per flow, ∼13.6× every flow's oracle FCT: rates settle near the sender floor (the $\eta$-hold's additive path back is too slow) and the fabric idles; a fixed maxBDP window sits between (1.52×). Scheduling can enforce *equality*; the max-min *allocation* — equal shares at full utilization — also requires a signal telling each sender how much to offer, and the absolute oracle-relative metric separates the outcomes (13.6× vs 1.06×). Per-flow queues also mean per-flow state at every port; the register needs three words. Our claim is thus scoped: for private reconstructions of an explicit fair rate, placement of the state is decisive — six sender-side designs fail while the identical law at the switch succeeds. It is not a formal impossibility for every endpoint law: contractive designs (AIMD-style convergence toward equal windows) escape the history argument in principle, though the AIMD-flavored schemes we measure (DCQCN, DCTCP) still show 1.6–2.3× (§5.3).

# 4 RDMA-RCP Design

The empirical and structural arguments of §3 motivate a design that (a) makes the *dominant* flows yield rather than merely nudging the victims, and (b) places a fair-rate *signal* at the switch, since the sender cannot recover the fair share from a load signal alone. We do not have the switch count flows and divide; instead each port runs a standard RCP control law on aggregate load and queue (below) whose fixed point *is* the max-min fair rate $C/N$. We package the most minimal such mechanism as an *operating mode* alongside stock HPCC on the same INT transport, selected per fabric or isolated slice (§4.3).

*Background: RCP.* The Rate Control Protocol (RCP) [9, 10] is a router-assisted congestion control in which each link advertises a *single* rate $R$ — the rate *each* flow should send at, not an aggregate budget — to all flows crossing it, holding no per-flow state. Routers stamp $R$ into passing packets and each flow sends at the minimum $R$ along its path. With $N$ flows each sending $R$, the controller drives $NR \to C$ (idle link $\Rightarrow$ $R$ rises; standing queue $\Rightarrow$ $R$ falls), so at equilibrium $R \approx C/N$ — the max-min (processor-sharing) share of the flows bottlenecked there, tracked *without counting* $N$: as flows arrive or leave, $R$ self-adjusts. RCP supplies §3's missing ingredient: a fair-share signal computed where flow count is implicitly observable — the link. RDMA-RCP runs this control law once per egress port and carries its $R$ in the INT header; a flow's path-minimum $R$ approximates network-wide max-min fairness (unfairness *ratio* within 1.2% of the oracle's; absolute per-flow FCT overheads are reported separately in §5.7), since a flow bottlenecked elsewhere contributes less load here and the loop hands the slack to the rest.

**Algorithm 1** RDMA-RCP switch — per egress port $j$

**State** $R_j$ (fair rate), $t_j$ (last update), $B_j$ (txByte snapshot)
**Const** $C_j$ link capacity; $\alpha$=0.4, $\beta$=0.226; $d$ control interval (≈ RTT); $R_{\text{floor}}$
**procedure** OnDequeue($pkt$, $j$) ▷ every packet leaving port $j$
  **if** $R_j$ uninitialised **then**
    $R_j \leftarrow C_j/2$; $t_j \leftarrow now$; $B_j \leftarrow txBytes_j$ ▷ conservative start
  **end if**
  **if** $now - t_j \geq d$ **then** ▷ one control interval elapsed
    $y \leftarrow (txBytes_j - B_j) \cdot 8 \,/\, (now - t_j)$ ▷ observed load
    $q \leftarrow queueBytes_j \cdot 8$ ▷ backlog (bits)
    $R_j \leftarrow R_j \cdot \left(1 + (\alpha(C_j - y) - \beta\, q/d)/C_j\right)$ ▷ RCP update
    $R_j \leftarrow \text{clamp}(R_j, R_{\text{floor}}, C_j)$
    $t_j \leftarrow now$; $B_j \leftarrow txBytes_j$
  **end if**
  **if** $pkt.fairRate = 0$ **then** ▷ first hop on the path
    $pkt.fairRate \leftarrow R_j$
  **else**
    $pkt.fairRate \leftarrow \min(pkt.fairRate, R_j)$ ▷ path-min
  **end if**
  $txBytes_j \leftarrow txBytes_j + \text{sizeof}(pkt)$
**end procedure**

## 4.1 The mechanism

The entire protocol is Algorithms 1 and 2. The switch maintains one scalar fair rate per egress port and stamps the path-minimum into a single header field; the sender reads that field from the returning ACK and adopts it. No per-flow switch state, no per-hop record, no receiver changes.

*At the switch (Alg. 1).* Each egress port keeps a single scalar fair rate $R_j$, refreshed once per control interval $d$ — a fabric-wide constant set to the maximum base RTT, 21.4 $\mu$s in all our experiments — by the standard RCP rule $R \leftarrow R\,(1 + (\alpha(C - y) - \beta q/d)/C)$, where $y$ is the load observed over the last interval and $q$ the queue backlog. The term $\alpha(C - y)$ chases spare capacity (positive while the link is under-utilized, pushing $R$ up) and $\beta q/d$ brakes when a standing queue forms (pulling $R$ down); they balance at $y \approx C$, $q \approx 0$, which is the $R \approx C/N$ fixed point above. We use the RCP defaults $\alpha = 0.4$, $\beta = 0.226$ (untuned; the result is insensitive to them, §5.9). The

**Algorithm 2** RDMA-RCP sender (receiver echoes the header unchanged)

1: **on** QP-create: $rate \leftarrow R_{\text{start}}$ ▷ cold-start; distinct from $R_{\text{floor}}$

2: **procedure** OnAck($a$)
3: **if** $a.fairRate = 0$ **then return** ▷ no signal yet
4: **end if**
5: $rate \leftarrow \text{clamp}(a.fairRate, R_{\text{floor}}, NICrate)$ ▷ adopt path-min fair share
6: **end procedure**

port's entire state is three numbers — $R_j$, the last update time, and a byte-counter snapshot — with *no* per-flow data.

*On the wire.* The FS INT mode (`IntHeader::FS`) carries one 64-bit field, `fairRate`; the serialized cost is 8 bytes per packet *independent of hop count* (vs. 8 bytes *per hop* for stock HPCC). Each switch min-aggregates $R_j$ into this field (first hop initialises it), so the receiver sees the path-minimum fair rate and echoes the header back unchanged — no receiver changes.

*At the sender (Alg. 2).* The new ACK handler `HandleAckFs` reads `fairRate` from the returning ACK and sets the sending rate to it, clamped to $[R_{\text{floor}}, NICrate]$. That single assignment is the whole sender control loop. Two parameters are deliberately distinct: the cold-start rate $R_{\text{start}}$ (1 Gbps; sent before the first ACK) and the adopted-rate floor $R_{\text{floor}}$ (100 Mbps; `fs_min_rate`) — conflating them turns $N \le C/R_{\text{start}}$ into a hard fan-in bound (measured in §5.11). ACKs are per-QP, in order, on a fixed reverse path, so the newest write wins; reordering multipath would need an epoch.

## 4.2 Startup smoothing

Two cold-start details: the switch initializes $R = C/2$ (not $C$, else the first stamp is line rate), and the sender starts at $R_{\text{start}} = 1$ Gbps so the first RTT does not blast before any ACK returns; adopted rates may fall to $R_{\text{floor}}$ thereafter. A flow sees a path-complete fair-rate signal in one RTT; full convergence takes several control intervals (within 5% of fair share by ~0.6 ms, 1% by 2.3 ms; §5) — fast and path-length-independent, though not literally one-RTT.

## 4.3 Additive integration and coexistence

RDMA-RCP is `cc_mode == 11`. The mode-3 (HPCC) and mode-10 (PINT) wire formats and switch/sender code paths are completely untouched; we add a fourth case to the INT header mode and its serialization, the switch and sender mode-dispatch, and one branch in the validation driver. The driver also forces the QP window to 0 in FS mode (rate-only control; §5.8 shows why a NIC-scaled window must not remain). In the shipped `cc_mode 11` the INT wire format is a global choice — one mode per fabric. To test per-class deployment we added `cc_mode 12`: stock HPCC's 42-byte wire layout (stock packets byte-for-byte unchanged) with per-packet class dispatch; the FS class reuses the record's first eight bytes and pays stock's full header cost, conservative for our claim. Both regression gates hold (all-stock ≡ `cc_mode 3` exactly; all-FS ≡ `cc_mode 11`'s 1.005×, zero PFC).

*Coexistence is not free (negative result).* On the $N$=4 parking lot with the long flow in the RCP class and the crosses in the stock class, *neither class gets its fair share*: rate-only RCP overshoots (21.6 vs 12.5 Gbps fair, stock squeezed to 11.1; 0.51×), the windowed variant under-shoots (6.2 Gbps, 2.53×), and a separate priority queue does not help (HPCC keeps its queue near zero and forfeits round-robin turns) — non-compliant flows are invisible to $R$'s inference exactly as flow count is invisible to a sender.

We then tested the obvious remedy: a weighted-DRR egress scheduler (byte-deficit DRR, `dwrr_weights`) reserving each class its exact 50% per-link entitlement. *It changes nothing* (0.511×): the scheduler never binds, because stock HPCC's INT still reports *port-global* bytes and queue — the stock class sees $u \approx 1$ from RCP traffic, holds low, and never offers its reserved share, which RCP then absorbs. Coexistence requires the reservation to extend into the *telemetry* — per-class $C$, $y$, $q$ — i.e. true slicing, or a separate fabric. All variants ship with the artifact (`run_mixed_mode.py`); PFC is zero throughout.

# 5 Evaluation

## 5.1 Methodology

All experiments use the deterministic parking-lot benchmark of §2, driven by our `hpcc-validation.cc`. Flows are 50 MB unless noted; sim time is 200 ms; the optimized binary is used throughout. We compare against stock HPCC (`cc_mode 3`, `multi_rate = false`), stock HPCC multi-rate, HPCC-PINT, and the `mb_mode` ablation suite. The simulator is the original HPCC ns-3 code base upgraded to a tagged ns-3.45 release — interface-level changes only; control logic and wire formats unchanged — verified by reproducing the original paper's baseline comparisons (HPCC vs. DCQCN, TIMELY, DCTCP, and PINT across WebSearch and FB-Hadoop workloads at 30–70% load on a 320-server fat-tree). The full reproduction record ships with the artifact,[3] and its verdict is that the upgraded code matches the original paper's central findings. Metrics, used consistently below: *unfairness ratio* = long-flow FCT / mean cross-flow FCT (oracle = 1.0); *oracle-relative FCT* $P_i$ = per-flow FCT / fluid max-min oracle FCT ($P_i < 1$ means a flow beat its oracle FCT by taking capacity the oracle assigns to others); *generalized unfairness* (heterogeneous cases) = worst-long / best-flow $P_i$; *raw long/short ratio* = mean long / mean short FCT (ECMP study); *slowdown* = FCT / same-size standalone FCT; *coflow JCT* = max FCT over a coflow. All "zero PFC" statements are with respect to the evaluated buffer/PFC configuration of §5.10.

## 5.2 Headline: N-sweep

Stock and multi-rate HPCC are starved across $N$ = 2–4 (penalty 1.90–1.96× for stock, 1.44–1.75× for multi-rate), then break entirely at $N \ge 5$, where the per-hop INT record overflows the `maxHop` cap (the apparent 0.51× there is the overflow artifact, not a real allocation). RDMA-RCP is flat at 1.003–1.007× across the entire sweep — its one-field format does not grow with path length. HPCC-PINT, carrying one compressed utilization field, inherits load-based

[3] Reproduction record and methodology: https://github.com/UNOCourseDemo/hpcc-fs/tree/main/examples/hpcc/repro-verification. The raw per-run results (554 MB compressed) are archived at https://1drv.ms/f/c/6052297178cce52b/IgAsSQ7gNfdhQrNejod27CpkAYQs8xZs4QtGc_KKVWZjQLs.

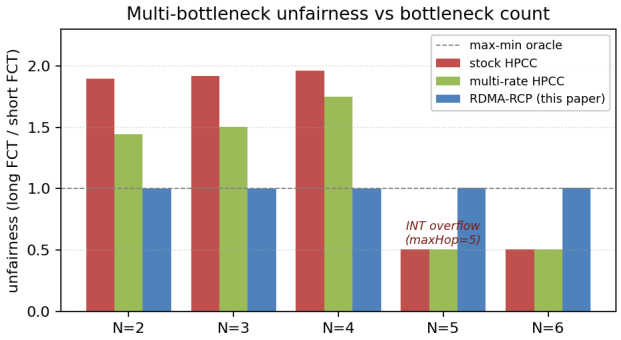



**Figure 2: Multi-bottleneck unfairness vs bottleneck count $N$. RDMA-RCP is flat $\approx 1.0$ across the entire sweep; stock and multi-rate HPCC fail at $N \leq 4$ and break at $N \geq 5$ (INT-overflow regime).**

**Table 1: Unfairness ratio and PFC pauses on the parking lot: every *evaluated* scheme fails multi-bottleneck max-min; all worsen with $N$.**

| scheme | $N$=2 | $N$=3 | $N$=4 | PFC |
|---|---|---|---|---|
| DCQCN | 1.70× | 1.90× | 2.32× | 6–34 |
| TIMELY | 1.83× | 1.96× | 2.01× | 72–156 |
| DCTCP | 1.61× | 1.72× | 1.78× | 256–280 |
| HPCC | 1.90× | 1.92× | 1.96× | 0 |
| RDMA-RCP | **1.003**× | **1.003**× | **1.005**× | 0 |

control and still suffers ~1.8× (1.77–1.88×) — the fix requires a fair-share signal, not a smaller INT footprint. All RDMA-RCP runs have zero PFC events.

## 5.3 The failure spans our evaluated stack

Is multi-bottleneck unfairness HPCC-specific? We ran every scheme in our stack on the same benchmark with harness parity (window-based schemes keep their window; DCQCN, TIMELY, DCTCP run windowless per their reference configurations — DCTCP as a datacenter-TCP baseline, not an RDMA scheme). Table 1: every scheme *in our evaluated stack* is unfair to the multi-bottleneck flow and worsens with bottleneck count — DCQCN worst at $N$=4 (2.32×). HPCC is the norm here, not an outlier; only the RCP mode reaches max-min, and only HPCC and the RCP mode hold zero PFC. Untested controllers (PowerTCP, Bolt, On-Ramp, Swift) are outside the claim; a same-benchmark comparison is future work.

## 5.4 When max-min is the objective: ring-collective coflow JCT

Collective communication motivates multi-bottleneck fairness: a step completes only when its *slowest* flow completes (JCT = max FCT), and the slowest flows are precisely the cross-pod, multi-bottleneck ones stock HPCC starves. We run a 16-flow ring coflow — the traffic of one communication phase of a ring allreduce (each host sends 50 MB to its ring successor: 4 inter-pod, 4 intra-pod, 8 same-edge segments) over all hosts of the $k$=4 fat-tree, across the same five ECMP hash seeds. A full allreduce chains $2(N-1)$ such phases; if later phases see similar contention and no compute overlap, their *absolute* savings accumulate — we do not simulate the multi-phase collective. RDMA-RCP cuts JCT by 19.5–22.7% (mean 21.4%) at *every* seed, and makes it nearly placement-invariant (17.9–18.0 ms across seeds, vs. stock's 22.3–23.3). Eight added 2 MB background flows leave the gain intact (+20.6%) and themselves speed up ~3× (0.9–1.3 vs 2.2–4.3 ms) — a short flow's fate under near-winner-take-all is a placement lottery; max-min removes it. A header-size control rules out the confound of RDMA-RCP's smaller INT record: re-running the coflow with the RCP class padded to stock's 42-byte layout (§4.3 transport, all flows FS) still gives 19.0 ms vs stock's 23.3 — 18% under byte-identical headers, vs 23% with the 8-byte field. The gain is fairness, not header thinness. The 8-flow ECMP workload, read as a coflow, shows the same: makespan improves 9.8–26.5% across seeds.

## 5.5 Structurally different topologies

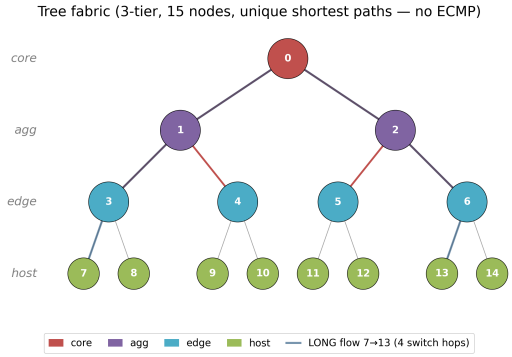
*Tree fabric (3-tier, no ECMP)..* A 3-tier tree with unique shortest paths (15 nodes; 25 Gbps inter-switch, 100 Gbps host; Figure 3a). Stock penalty 1.36×, RDMA-RCP **1.003**×, PFC = 0.

*$k$ = 4 fat-tree (with ECMP)..* The canonical fat-tree (20 switches + 16 hosts; Figure 3b), hash-based ECMP; 4 inter-pod (5-switch) plus 4 intra-pod (3-switch) flows. On the *oracle-normalized penalty* (long-path vs. short-path slowdown, each against the max-min oracle, removing placement effects): stock 1.34×, RDMA-RCP **1.001**×. The *raw* long/short ratio stays ~1.32× under RDMA-RCP— not CC unfairness but placement: ECMP hashes some flows onto underloaded cores, inflating the raw ratio for any controller. PFC = 0 for both; path-min aggregation is path-invariant, so ECMP needs no design changes. *Hash-seed sensitivity.* Because a single deterministic run fixes one placement, we re-ran the workload across *twenty* ECMP hash seeds (an additive `ecmp_seed_offset` knob shifts every switch's hash seed; the default preserves stock behavior), reporting the mean-long/mean-short raw ratio. Under stock HPCC that ratio varies 0.96–1.40× across seeds (mean 1.14) *with no change to the congestion controller* — direct evidence that the raw spread is placement-dominated. Under RDMA-RCP the mean is 1.04 and the median 1.001: fifteen of twenty seeds sit within 3% of parity, and the five residuals (1.16–1.33×) are placements whose hash collides the long flows onto shared cores, where equal FCTs are not the oracle allocation either. At the default seed the contended case drops 1.338× → 1.001× (seed-robust; `run_ecmp_seeds.py`).

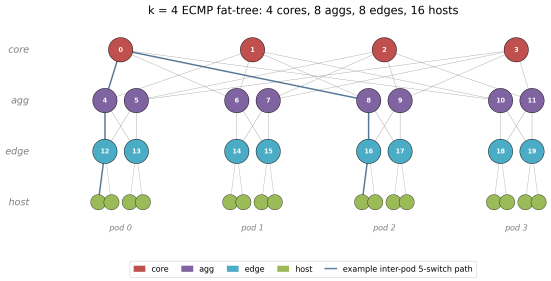
## 5.6 Robustness across asymmetric workloads

Across six perturbations (Figure 4) RDMA-RCP holds within 1.0 ± 0.012; the worst stock case (3.53×, small long flow) drops to 1.012×.

*Start-time jitter.* Jittering all five starts by $U[0, 1]$ ms (five seeds): stock 1.96–1.99×, RDMA-RCP 1.008–1.017×. On the $N$=1 control the pairwise spread persists at 1.10–1.39× across 10 $\mu$s–5 ms offsets with its *direction* flipping by arrival order — the $\eta$-hold lottery, not synchronization — while RDMA-RCP equalizes the pair at every offset.




**(a) Tree fabric — unique shortest paths, no ECMP.**




**(b) $k = 4$ ECMP fat-tree (4 pods).**

**Figure 3: Two structurally different topologies used to test generalization beyond the parking lot. (a) 3-tier tree fabric, 15 nodes; the highlighted line is the long flow $7 \rightarrow 13$, crossing four switches. (b) Canonical $k = 4$ fat-tree (4 cores, 8 aggs, 8 edges, 16 hosts in 4 pods); the highlighted line is a representative inter-pod 5-switch path.**

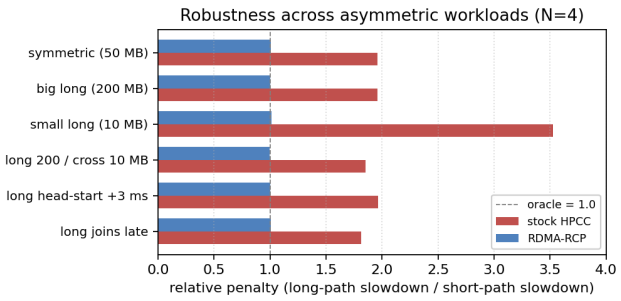



**Figure 4: Robustness across asymmetric-size and staggered-start variants at $N = 4$. RDMA-RCP is within $1.0 \pm 0.012$ across all six scenarios, including the small-long worst case where stock reaches 3.53×.**

## 5.7 Heterogeneous and dynamic contention

The uniform one-competitor parking lot is deliberately minimal, so we stress its assumptions (Table 2; `run_stress_matrix.py`): heterogeneous capacities, unequal competitor counts, two overlapping multi-bottleneck flows, and cross-flow churn. Each flow is scored against a generic event-driven progressive-filling oracle on the actual topology and schedule. A ratio alone can mask a uniform miss, so the table reports both the *generalized unfairness* (worst-long / best-flow $P_i$, reducing to the headline ratio in the uniform case) and the absolute per-flow range of $P_i$. Stock HPCC's unfairness grows with contention richness — worst with two overlapping long flows (2.87×) and churn (2.23×), its $P_i$ scattered 0.41–1.34 (some flows racing ahead of their oracle share while the longs lag) — while RDMA-RCP holds 1.001–1.007× with $P_i \in [1.05, 1.13]$: *every* flow lands within 5–13% of its own oracle FCT, a uniform convergence overhead, and mean bottleneck utilization is *higher* than stock in all four scenarios (0.65–0.78 vs 0.51–0.70). PFC = 0 everywhere; 4–8× RTT spreads applied to the mode itself leave the ratio at 1.003–1.008×. Jointly stressing the mode — churn plus a 4× RTT spread plus $d$ misconfigured 4× too small, at once — leaves the ratio at 1.000× while the absolute $P_i$ rise to 1.21–1.39: the compound stress costs uniform slowdown, which the absolute metric exposes and the ratio alone would hide (`run_round3.py`). (INT admits only the discrete rates {25, 50, 100, 200, 400} Gbps; heterogeneity is exercised within that set.)

**Table 2: Beyond the uniform parking lot: generalized unfairness and the absolute per-flow range of oracle-relative FCT $P_i$. RDMA-RCP holds ≈1.0× with every flow within 5–13% of its oracle FCT; PFC = 0 throughout.**

| | stock HPCC | | RDMA-RCP | |
|---|---|---|---|---|
| scenario | unf. | $P_i$ range | unf. | $P_i$ range |
| hetero cap. [25,50,25,100] G | 1.82× | 0.64–1.16 | **1.003**× | 1.06–1.06 |
| unequal comp. [3,1,2,1] | 1.41× | 0.84–1.18 | **1.001**× | 1.06–1.06 |
| two overlapping longs | 2.87× | 0.41–1.19 | **1.007**× | 1.05–1.06 |
| cross-flow churn | 2.23× | 0.60–1.34 | **1.002**× | 1.06–1.13 |

## 5.8 Ablation: windows must scale with the fair rate

RDMA-RCP defaults to rate-only operation — the per-flow window cap is disabled (§4). Table 3: re-enabling HPCC's `var_win` degrades the penalty to 1.5–2.5×, worsening with path length — that window is sized from the NIC line rate, not the multi-hop bottleneck. The problem is the window's *scale*, not windowing: $cwnd = R \cdot \text{baseRTT}$ (`fs_rcp_window`) matches rate-only fairness while bounding in-flight data — a cap must track $R$, not the NIC.

## 5.9 Parameter sensitivity

We use the standard RCP defaults ($\alpha = 0.4$, $\beta = 0.226$) untouched. Sweeping each parameter one at a time at $N = 4$ — $\alpha \in [0.2, 0.6]$, $\beta \in [0.1, 0.4]$, startup $R_0/C \in [0.25, 1.0]$, $minRate \in [100, 2000]$ Mb/s — the penalty stays within $[1.004, 1.008]$× with zero PFC throughout (even $R_0 = C$: the cold-start absorbs the first interval);

**Table 3: Window ablation: the NIC-scaled window breaks fairness; a fair-rate-scaled canonical RCP window preserves it (all zero PFC).**

| $N$ | rate-only (default) | NIC-scaled window | $cwnd = R \cdot \mathrm{RTT}$ |
|---|---|---|---|
| 2 | **1.003×** | 1.50× | **1.004×** |
| 3 | **1.003×** | 2.00× | **1.004×** |
| 4 | **1.005×** | 2.49× | **1.006×** |

the parameters are not load-bearing in this family (§5.7 adds a joint stress). The interval $d$ tolerates misconfiguration asymmetrically: 2–4× too large or 2× too small stays 1.003–1.011×; 4× too small degrades to 1.212× (updates outrun feedback) — still far below every baseline, PFC zero.

## 5.10 Queueing and PFC

For context, every run uses the validated HPCC buffer model: 4 MB shared buffer with dynamic PFC thresholds, ECN marking $k_{\min}/k_{\max}/p_{\max}$ = 100 KB/400 KB/0.2 at 25 Gbps (400/1600 KB at 100 Gbps), 1000 B payload, 1–2 $\mu$s link delays. With the smoothed startup (§4.2), RDMA-RCP produces **zero PFC events** on every workload in the $N$-sweep and the robustness matrix. ECN marking remains; queue length is bounded (peak ≈46 KB at $N = 4$, versus ≈118 KB for stock HPCC on the same workload); no flow loss. Do-no-harm holds throughout: the RDMA-RCP additions (implementation name `HPCC-FS`, `cc_mode 11`) only add code, and stock `cc_mode 3` smoke tests and $N$-sweep outputs are byte-identical to the pre-change baseline.

## 5.11 High fan-in and idle-port staleness

Two stress cases target the update rule itself (`run_round4.py`). *Incast fan-in, and the rate floor:* $S$ senders to one 25 Gbps receiver link, 200 KB each, $S$ up to 128. Fan-in exposes why $R_{\text{start}}$ and $R_{\text{floor}}$ must not be conflated: with the floor tied to the 1 Gbps cold-start (our initial implementation), $S \le C/R_{\text{floor}} = 25$ is a *hard feasibility bound* — at $S$=64 every sender is clamped above its 0.39 Gbps fair share, and PFC enforces the sharing the endpoint is prohibited from adopting: 2,045 pauses, 142 ms cumulative paused time. With the floor decoupled (100 Mbps), the overload disappears: **PFC = 0 through** $S$=128, with per-sender rates reaching 182 Mbps at $S$=64 — well below the old clamp. The residual cost is convergence: mean FCT 8.8 vs stock HPCC's 4.5 ms at $S$=64 (16.0 vs 8.9 at $S$=128), while HPCC itself takes 64 pauses (36.5 ms paused) at $S$=64. Extreme fan-in remains stock HPCC's home turf on raw FCT; the rate mode's behavior there is now graceful, not pathological. The floor never binds when fair shares exceed it; every fairness result reproduces exactly. *Idle-port staleness:* $R$ updates only on dequeue, so a port that goes idle freezes its last $R$. We congest a port with four flows, let it idle for $g \in \{0.1, 1, 10, 100\}$ ms, then release a single 1 MB probe: its FCT is 0.373 ms at *every* gap — faster than a fresh port's 0.713 ms ($R_0 = C/2$ ramp) — because the drain tail's final dequeues carry $R$ back to ≈$C$ before the port empties. The frozen value is benign after *naturally draining* flows — the tested trajectory; an abrupt halt (failure, reroute) could freeze a low $R$, which we do not model; idle aging (reset after $k \cdot d$ without dequeues) would cover it.

## 5.12 HPCC's home turf: competitive FCT, lower queue

What does the RCP mode cost on *single*-bottleneck workloads — HPCC's home turf? We measure the validated incast benchmark (3 senders → 1 receiver over 25 Gbps; 12 flows, 100–400 KB), sweeping the sender's startup rate. Two things stand out: RDMA-RCP holds far less queue than HPCC at every startup (38–64 KB vs 143 KB), and HPCC's only lead — mean FCT at the conservative 1 Gbps startup (297.5 vs 258.6 $\mu$s, ∼15%) — closes and reverses at 8 Gbps (242.0 vs 258.6 $\mu$s at one-quarter the queue, zero PFC). The 8 Gbps point leaves the multi-bottleneck headline undisturbed (1.003–1.004×, PFC = 0); the headline runs use 1 Gbps. On these workloads the fairness mode costs little-to-no FCT and holds less queue. Before the first fair-rate ACK a sender emits at most *startup* × baseRTT (≈2.7 KB at 1 Gbps), and the $cwnd = R \cdot \mathrm{RTT}$ variant (§5.8) restores a hard per-flow cap thereafter; the fan-in limit of this startup bound is measured directly in §5.11. RDMA-RCP still lacks HPCC's per-hop, window-bounded reaction — §5.11 measures the $S$=64 boundary — and recovering it while keeping fairness is the hybrid's target (§7).

## 5.13 Per-flow rate trace, before vs after

The before-vs-after rate trace (Figure 5) is the cleanest qualitative result: collapse to ∼0.4 Gbps under stock, lock-in at fair share under RDMA-RCP.

## 5.14 Scope of the headline claim

The ≈1.0× headline is a statement about the *congestion-control component* of multi-bottleneck unfairness on contention benchmarks — not a claim of universal FCT improvement on production workloads. Five first-class limits. (i) *Mixed short-flow workloads* do not exhibit the penalty we target: under a WebSearch distribution stock HPCC's oracle-normalized penalty is already 1.001× (RDMA-RCP: 0.581× — short flows run *ahead* of their max-min share: policy deviation, not superiority), and on mean slowdown RDMA-RCP trades ∼15% off multi-bottleneck flows for ∼45% on short paths. Short-flow-FCT workloads [4, 18] may prefer stock HPCC; max-min matters when multi-bottleneck flows gate a job (§5.4). Per-service selection needs per-class link or telemetry isolation (§4.3: an unsliced port serves neither class); the deployment unit is a fabric or slice dedicated to max-min traffic, not a global replacement. (ii) *Fabric scale*: our $k = 6, 8$ scaling workload does not form persistent inter-pod contention (both penalties <1), so we added a *saturating* $k$=6 case — 108 identical 20 MB inter-pod flows, every stage at capacity under ECMP (`run_round4.py`): coflow makespan 42.9 → 36.7 ms (−14%; −11% with the INT header padded to stock size), Jain 0.910 → 0.924, PFC = 0 at 45 switches. Both schemes traverse *identical* ECMP placements (same hash, seeds, 5-tuples), so the placement penalty (2.9× vs 3.4× the 12.8 ms perfect-balance fluid bound) is common and the delta is controller-attributable; a path-resolved oracle at scale needs path instrumentation, so we claim no oracle-relative fairness here ($k$=8: future work). (iii) *ECMP placement*: a raw ∼1.3× long/short spread from hash placement

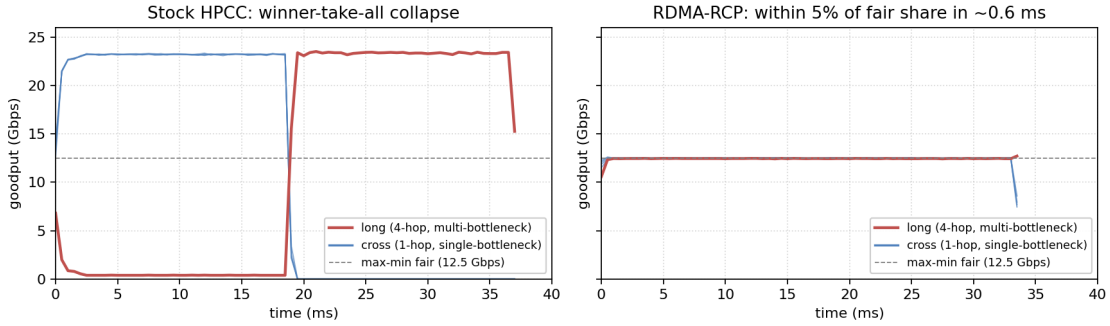



**Figure 5: Per-flow goodput at $N = 4$, stock HPCC (left) vs RDMA-RCP (right). Stock collapses the long flow to ~0.4 Gbps while cross flows hold ~23 Gbps for 18 ms (near-winner-take-all). RDMA-RCP converges all five flows to within 5% of the max-min fair share (12.5 Gbps) in ~0.6 ms and to within 1% by ~2.3 ms.**

remains; RDMA-RCP removes the CC component only. (iv) *Coexistence*: the modes do not share a link fairly even under DRR reservation (§4.3); RDMA-RCP is a fabric-wide mode. (v) *Extreme fan-in*: past $S \approx 64$ the rate mode completes behind stock HPCC (§5.11); combined with (iv), incast-heavy services belong on the stock side of the slice boundary — a real constraint on which classes a rate-mode fabric can host. (vi) *Fairness unit*: max-min here is per QP/flow — a job opening $m$ QPs obtains $m$ shares; job-level fairness would need weighted rates ($R_i = w_i R$) or QP-count policing, which we do not evaluate.

## 6 Related Work

*Datacenter transport.* DCTCP [3], DCQCN [24], TIMELY [17], and Swift [14] trade buffering for latency via ECN- and delay-based control; Homa [18] and pFabric [4] prioritize short flows; HPCC [15] drives control from per-hop INT, with On-Ramp [16] and PowerTCP [1] extending the precise-signal philosophy. Across this line the target is single-bottleneck FCT, tail latency, or queueing; §5.3 shows every member we evaluate fails multi-bottleneck fairness.

*In-network telemetry.* INT [13] lets the data plane stamp per-hop state into packets on programmable pipelines, and HPCC was among the first to drive congestion control directly from it. The cost is header overhead, which later work minimizes: HPCC-PINT [23] compresses the per-hop record into a single probabilistic field, and Poseidon [22] restructures INT-based control for deployability on commodity hardware. RDMA-RCP continues this minimal-telemetry direction but changes *what* the field carries: an explicit fair *rate*, min-aggregated along the path — one 64-bit field, like PINT, yet a decision rather than a measurement.

*Explicit-rate and switch-assisted control.* Having the switch compute and return a sending rate is a classic idea. ATM ABR explicit-rate schemes such as ERICA [11] computed a fair share per port decades ago; XCP [12] decomposed switch feedback into separate efficiency and fairness controllers; and RCP [9] reduced this to a single fair-rate scalar per link with no per-flow state, packet-stamped, with senders taking the path minimum. RCP is the closest mechanism to ours: RDMA-RCP adopts its update rule but situates it as a *minimal fairness signal injected into an existing INT load-signaling system* rather than a standalone protocol, keeping HPCC's single-rate sender and per-hop transport. Prior RCP work established explicit-rate fairness in general fabrics; what had not been measured is whether deployed INT-based RDMA stacks exhibit the multi-bottleneck gap, and whether a telemetry-carried fair rate closes it *inside* such a stack — that measurement is this paper's contribution. RDMA-RCP's per-port update and path-min aggregation *are* RCP's; the endpoint differs (rate-only by default; the canonical window performs equally, Table 3), the control interval is a fabric constant, updates are event-driven, and the port applies $R_0 = C/2$ and a floor. Over deploying RCP as a separate protocol, RDMA-RCP adds packaging, not control: one field on the INT transport already running, a byte-identical stock path, per-mode selection — no second stack.

*In-switch fairness.* Fair queueing [8], DRR [20], CSFQ [21], and AFD [19] enforce fairness in the scheduler; §3 measures the boundary — scheduling equalizes only what senders offer, and without a rate signal the equal outcome is far from the max-min allocation. RDMA-RCP instead *advertises* one fair rate per port — no per-flow tracking, no extra queues — implementable on the match-action pipelines [7] INT already assumes.

*Closely related switch-assisted DC schemes.* Bolt [5] accelerates HPCC's loop with sub-RTT per-flow switch assistance, targeting tail latency and ramp-up — orthogonal to our objective: its feedback carries load/supply, not a fair share, so the history argument of §3 applies to its senders too; a faster load-signal loop converges faster to the same equilibrium. We leave a head-to-head to future work.

## 7 Discussion

*Hybrid path.* RDMA-RCP inherits RCP's fair, low-queue character without HPCC's single-flow precision (§5.12); a future hybrid caps HPCC at the fair rate, $\min(rate_{\text{HPCC}}, R)$, with a fair-rate-scaled window (§5.8).

*Scaling and hardware.* At $k = 6, 8$ the fabric's $(k/2)^2$ cores spread inter-pod flows so persistent contention does not materialize for our scaling workload (both schemes' penalty <1; RDMA-RCP adds no overhead, 0 PFC); the saturating case is in §5.14. On hardware

we offer a mapping, not a prototype: three registers per port ($R$, timestamp, byte snapshot); a dequeue-triggered fixed-point update with $1/C$ and $\beta/d$ precomputed (two multiplies, two subtractions, a clamp); and a 64-bit compare-and-write path-min per packet. Atomicity comes from the RMT stateful-ALU discipline: the timestamp compare and the $R$ update commit in one per-packet read-modify-write, so concurrent in-flight packets cannot double-apply an interval. Plausible widths: 32-bit $R$ (Mbps), byte counters, and wrap-safe ns timestamps; 24-bit queue depth; a Q16 multiplier with saturating clamp. Each piece has P4 precedent, but stage budget, dequeue-time queue-depth access, and quantization behavior under fixed-point arithmetic are unverified; a Tofino prototype remains future work.

*Limitations.* Three topology families; in-sim baselines only (DCQCN, TIMELY, DCTCP, multi-rate HPCC, PINT) — no Bolt/PowerTCP, no testbed; per-class coexistence requires link or telemetry isolation we do not design (§4.3); the ECMP study is a small controlled workload (though seed-swept), not a realistic mixed-size + ECMP combination. All numbers come from one simulator — but its control logic is the original HPCC implementation with interface-level modernization only, verified end-to-end by reproducing the original baselines, and the $N \leq 4$ regime sits within the 5-hop INT budget by construction; a belt-and-braces run on the unmodified legacy build remains future work; the result is insensitive to the RCP parameters (§5.9).

## 8 Conclusion

Multi-bottleneck unfairness spans every deployable RDMA scheme in our stack, and HPCC's instance is a stable, RTT-independent, near-winner-take-all equilibrium (~2×; 3.5× for small flows; unchanged over 10× longer backlogs). Six sender-side corrections fail — including an exact private mirror of RCP — and per-flow fair queueing equalizes without approaching the allocation (1.97× windowed; ~13× the oracle FCT windowless); a controlled fluid experiment shows why: with identical observations, law, and epochs, privately held explicit-rate state still freezes arrival-order asymmetry, while one shared per-link register erases that history. RDMA-RCP carries that register's value in one INT field — three words of mutable state per port — reaching penalty 1.005×, every flow within 5–13% of its oracle FCT in the heterogeneous matrix (21–39% uniform under compound stress), with zero PFC in every fairness experiment, rate-only or canonically windowed, converging sub-millisecond and independent of path length, with the stock HPCC code byte-for-byte unchanged. Coexistence on a shared link fails even under a DRR reservation (§4.3): the mode belongs on sliced telemetry or a separate fabric, not in free competition.

## References


[1] Vamsi Addanki, Maria Apostolaki, Manya Ghobadi, Stefan Schmid, and Laurent Vanbever. 2022. PowerTCP: Pushing the Performance Limits of Datacenter Networks. In *Proc. USENIX NSDI*. 51–70.

[2] Alibaba. 2019. HPCC reference simulator (ns-3). https://github.com/alibaba-edu/High-Precision-Congestion-Control. `int-header.h`: `static const uint32_t maxHop = 5`.

[3] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. 2010. Data Center TCP (DCTCP). In *Proc. ACM SIGCOMM*. 63–74.

[4] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. 2013. pFabric: Minimal Near-Optimal Datacenter Transport. In *Proc. ACM SIGCOMM*.

[5] Serhat Arslan, Yuliang Li, Gautam Kumar, and Nandita Dukkipati. 2023. Bolt: Sub-RTT Congestion Control for Ultra-Low Latency. In *Proc. USENIX NSDI*. 17 pages.

[6] Dimitri Bertsekas and Robert Gallager. 1992. *Data Networks* (2nd ed.).

[7] Pat Bosshart, Dan Daly, Glen Gibb, Martin Izzard, Nick McKeown, Jennifer Rexford, Cole Schlesinger, Dan Talayco, Amin Vahdat, George Varghese, and David Walker. 2014. P4: Programming Protocol-Independent Packet Processors. *ACM SIGCOMM Computer Communication Review* 44, 3 (2014).

[8] Alan Demers, Srinivasan Keshav, and Scott Shenker. 1989. Analysis and Simulation of a Fair Queueing Algorithm. In *Proc. ACM SIGCOMM*.

[9] Nandita Dukkipati. 2008. *Rate Control Protocol (RCP): Congestion Control to Make Flows Complete Quickly*. Ph. D. Dissertation. Stanford University.

[10] Nandita Dukkipati and Nick McKeown. 2006. Why Flow-Completion Time is the Right Metric for Congestion Control. *ACM SIGCOMM Comput. Commun. Rev.* 36, 1 (2006), 59–62.

[11] Shivkumar Kalyanaraman, Raj Jain, Sonia Fahmy, Rohit Goyal, and Bobby Vandalore. 2000. The ERICA Switch Algorithm for ABR Traffic Management in ATM Networks. *IEEE/ACM Transactions on Networking* 8, 1 (2000).

[12] Dina Katabi, Mark Handley, and Charlie Rohrs. 2002. Internet Congestion Control for Future High Bandwidth-Delay Product Environments. In *Proc. ACM SIGCOMM*. 89–102.

[13] Changhoon Kim, Anirudh Sivaraman, Naga Katta, Antonin Bas, Advait Dixit, and Lawrence J. Wobker. 2015. In-band Network Telemetry via Programmable Dataplanes. In *Proc. ACM SIGCOMM (Industrial Demo)*.

[14] Gautam Kumar, Nandita Dukkipati, Keon Jang, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. 2020. Swift: Delay is Simple and Effective for Congestion Control in the Datacenter. In *Proc. ACM SIGCOMM*.

[15] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yu. 2019. HPCC: High Precision Congestion Control. In *Proc. ACM SIGCOMM*. 44–58.

[16] Shiyu Liu, Ahmad Ghalayini, Mohammad Alizadeh, Balaji Prabhakar, Mendel Rosenblum, and Anirudh Sivaraman. 2021. Breaking the Transience-Equilibrium Nexus: A New Approach to Datacenter Packet Transport. In *Proc. USENIX NSDI*.

[17] Radhika Mittal, Vinh The Lam, Nandita Dukkipati, Emily Blem, Hassan Wassel, Monia Ghobadi, Amin Vahdat, Yaogong Wang, David Wetherall, and David Zats. 2015. TIMELY: RTT-based Congestion Control for the Datacenter. In *Proc. ACM SIGCOMM*. 537–550.

[18] Behnam Montazeri, Yilong Li, Mohammad Alizadeh, and John Ousterhout. 2018. Homa: A Receiver-Driven Low-Latency Transport Protocol Using Network Priorities. In *Proc. ACM SIGCOMM*.

[19] Rong Pan, Lee Breslau, Balaji Prabhakar, and Scott Shenker. 2003. Approximate Fairness through Differential Dropping. *ACM SIGCOMM Computer Communication Review* 33, 2 (2003).

[20] M. Shreedhar and George Varghese. 1995. Efficient Fair Queuing using Deficit Round Robin. In *Proc. ACM SIGCOMM*.

[21] Ion Stoica, Scott Shenker, and Hui Zhang. 1998. Core-Stateless Fair Queueing: Achieving Approximately Fair Bandwidth Allocations in High Speed Networks. In *Proc. ACM SIGCOMM*.

[22] Weitao Wang, Masoud Moshref, Yuliang Li, Gautam Kumar, T. S. Eugene Ng, Neal Cardwell, and Nandita Dukkipati. 2023. Poseidon: Efficient, Robust, and Practical Datacenter CC via Deployable INT. In *Proc. USENIX NSDI*.

[23] Liron Schiff Yang, Ran Ben Basat, Michael Mitzenmacher, Maria Apostolaki, Aviv Klein, Erez Saito, and Avi Mendelson. 2020. PINT: Probabilistic In-band Network Telemetry. In *Proc. ACM SIGCOMM*. 662–677.

[24] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. 2015. Congestion Control for Large-Scale RDMA Deployments. In *Proc. ACM SIGCOMM*. 523–536.